

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-212БВ-25

Студент: Волгин М.Е

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 05.10.26

Москва, 2026

Постановка задачи

Вариант 2.

Необходимо составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними посредством неименованного канала `pipe`.

Родительский процесс должен создать дочерний процесс. Первой строкой пользователь вводит в консоль родительского процесса имя файла, которое передаётся дочернему процессу при его запуске. Родительский и дочерний процессы должны быть представлены отдельными программами.

После запуска дочернего процесса пользователь вводит команды вида:

число число число<newline>

Количество чисел в одной строке может быть произвольным. Числа имеют тип `float`.

Родительский процесс передаёт введённые данные дочернему процессу через неименованный канал. Читающий конец канала связывается со стандартным входным потоком дочернего процесса. Дочерний процесс считывает полученные числа, вычисляет сумму чисел каждой строки и записывает результат в созданный им файл.

В процессе выполнения программы необходимо обрабатывать возможные ошибки системных вызовов.

Общий метод и алгоритм решения

Программа состоит из двух отдельных частей: `parent` и `child`.

Родительский процесс считывает имя файла, создаёт неименованный канал с помощью `pipe()` и порождает дочерний процесс вызовом `fork()`. В дочерней ветке читающий конец канала перенаправляется на `STDIN_FILENO` с помощью `dup2()`, после чего через `execve()` запускается программа `child`. Имя выходного файла передаётся ей через `argv`.

Родитель считывает команды пользователя и передаёт их через канал дочернему процессу. После завершения ввода канал закрывается, а родитель ожидает завершения дочернего процесса с помощью `waitpid()`.

Дочерняя программа открывает переданный файл с помощью `open()`, считывает данные через `read()`, разбирает числа типа `float` и вычисляет сумму для каждой строки. Полученный результат записывается в файл. После завершения работы файл закрывается.

Дополнительный канал `pipe2` не используется, так как результат по условию варианта записывается непосредственно в файл.

Использованные системные вызовы:

`pid_t fork(void);`

Создаёт дочерний процесс.

`int pipe(int pipefd[2]);`

Создаёт неименованный канал. `pipefd[0]` используется для чтения, `pipefd[1]` — для записи.

```
int dup2(int oldfd, int newfd);
```

Используется для перенаправления читающего конца канала на стандартный ввод дочернего процесса.

```
int execve(const char *filename, char *const argv[], char *const envp[]);
```

Заменяет исполняемую программу дочернего процесса на программу child.

```
ssize_t read(int fd, void *buf, size_t count);
```

Используется для чтения пользовательского ввода и данных из канала.

```
ssize_t write(int fd, const void *buf, size_t count);
```

Используется для передачи данных через канал и записи результатов.

```
int open(const char *pathname, int flags, mode_t mode);
```

Используется дочерним процессом для создания и открытия выходного файла.

```
int close(int fd);
```

Закрывает файловые дескрипторы.

```
pid_t waitpid(pid_t pid, int *status, int options);
```

Используется родительским процессом для ожидания завершения дочернего процесса.

Алгоритм работы программы

1. Родитель считывает имя выходного файла.
2. Создаётся канал pipe.
3. С помощью fork() создаётся дочерний процесс.
4. В дочернем процессе канал связывается с STDIN_FILENO через dup2().
5. С помощью execve() запускается программа child.
6. Родитель передаёт пользовательский ввод через канал.
7. Дочерний процесс считывает и разбирает числа типа float.
8. Для каждой строки вычисляется сумма.
9. Результат записывается в файл.
10. Родитель ожидает завершения дочернего процесса через waitpid().

Код программы

parent.c

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int write_all(int fd, const char* buffer, size_t size) {
    size_t written = 0;
```

```
while (written < size) {
    ssize_t result = write(fd, buffer + written, size - written);

    if (result <= 0) {
        return -1;
    }

    written += result;
}

return 0;
}

int main(void) {

    char filename[256];
    int index = 0;

    if (write_all(STDOUT_FILENO, "Enter the filename: ", 20) == -1) {
        return 1;
    }

    while (index < 255) {
        char symbol;

        ssize_t result = read(STDIN_FILENO, &symbol, 1);

        if (result < 0) {
            write_all(STDERR_FILENO, "read error.\n", 12);
            return 1;
        }

        if (result == 0 || symbol == '\n') {
            break;
        }

        filename[index] = symbol;
        index++;
    }

    filename[index] = '\0';

    int pipefd[2];

    if (pipe(pipefd) == -1) {
        write_all(STDERR_FILENO, "pipe error.\n", 12);
        return 1;
    }

    pid_t pid = fork();
```

```
if (pid < 0) {
    write_all(STDERR_FILENO, "Fork failed.\n", 13);
    close(pipefd[0]);
    close(pipefd[1]);

    return 1;
}

if (pid == 0) {
    close(pipefd[1]);

    if (dup2(pipefd[0], STDIN_FILENO) == -1) {
        write_all(STDERR_FILENO, "dup2 error.\n", 12);
        close(pipefd[0]);
        return 1;
    }

    close(pipefd[0]);

    char* argv[] = {"/child", filename, NULL};

    char* envp[] = {NULL};

    execve("/child", argv, envp);

    write_all(STDERR_FILENO, "execve error.\n", 14);
    return 1;
}

close(pipefd[0]);

if (write_all(STDOUT_FILENO, "Enter number: (Ctrl+D) \n", 24) == -1) {
    close(pipefd[1]);
    return 1;
}

while (1) {
    char symbol;

    ssize_t result = read(STDIN_FILENO, &symbol, 1);

    if (result < 0) {
        write_all(STDERR_FILENO, "read error.\n", 12);
        close(pipefd[1]);
        return 1;
    }

    if (result == 0) {
        break;
    }

    if (write_all(pipefd[1], &symbol, 1) == -1) {
        write_all(STDERR_FILENO, "write error\n", 12);
        close(pipefd[1]);
        return 1;
    }
}
```

```

    }
}

if (close(pipefd[1]) == -1) {
    write_all(STDERR_FILENO, "close error.\n", 13);
    return 1;
}

int status;

if (waitpid(pid, &status, 0) == -1) {
    write_all(STDERR_FILENO, "waitpid error.\n", 15);
    return 1;
}

if (WIFEXITED(status)) {
    int child_status = WEXITSTATUS(status);

    if (child_status != 0) {
        write_all(STDERR_FILENO, "Child finished with error.\n", 27);
        return 1;
    }
} else {
    write_all(STDERR_FILENO, "Child terminated. \n", 20);
    return 1;
}

return 0;
}

```

child.c

```

#include <unistd.h>
#include <fcntl.h>

int write_all(int fd, const char* buffer, size_t size) {
    size_t written = 0;

    while (written < size) {
        ssize_t result = write(fd, buffer + written, size - written);

        if (result <= 0) {
            return -1;
        }

        written += result;
    }

    return 0;
}

int write_number(int fd, unsigned long long number) {
    char buffer[32];

```

```

int index = 0;

if (number == 0) {
    if (write_all(fd, "0", 1) == -1) {
        return -1;
    }

    return 0;
}

while (number > 0) {
    buffer[index] = '0' + (number % 10);
    number /= 10;
    index++;
}

while (index > 0) {
    index--;
    if (write_all(fd, &buffer[index], 1) == -1) {
        return -1;
    }
}

return 0;
}

int write_float(int fd, float number) {

    if (number < 0) {
        if (write_all(fd, "-", 1) == -1) {
            return -1;
        }
        number = -number;
    }

    number += 0.0000005f;

    unsigned long long integer_part = (unsigned long long)number;

    if (write_number(fd, integer_part) == -1) {
        return -1;
    }

    if (write_all(fd, ".", 1) == -1) {
        return -1;
    }

    float fraction = number - (float)integer_part;

    for (int i = 0; i < 6; i++) {
        fraction *= 10.0f;

        int digit = (int) fraction;

        char symbol = '0' + digit;

        if (write_all(fd, &symbol, 1) == -1) {
            return -1;
        }
    }
}

```

```

    fraction -= digit;
}

return 0;
}

int main(int argc, char* argv[]) {

    if (argc < 2) {
        write_all(STDERR_FILENO, "No filename.\n", 13);
        return 1;
    }

    int file_fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (file_fd == -1) {
        write_all(STDERR_FILENO, "open error.\n", 12);
        return 1;
    }

    char buffer[256];

    float current_number = 0.0f;
    float sum = 0.0f;

    int sign_seen = 0;
    int invalid_line = 0;
    int in_number = 0;
    int sign = 1;
    int line_has_number = 0;
    int after_dot = 0;
    float fraction_divisor = 10.0f;

    while (1) {
        ssize_t bytes = read(STDIN_FILENO, buffer, sizeof(buffer));

        if (bytes < 0) {
            write_all(STDERR_FILENO, "read error.\n", 12);
            close(file_fd);
            return 1;
        }

        if (bytes == 0) {

            if ((sign_seen || after_dot) && !in_number) {
                invalid_line = 1;
            }

            if (in_number) {
                sum += current_number * sign;
                line_has_number = 1;
            }

            if (invalid_line) {
                write_all(STDERR_FILENO, "Invalid input.\n", 15);
            }
        }
    }
}

```

```

    } else if (line_has_number) {
        if (write_float(file_fd, sum) == -1) {
            write_all(STDERR_FILENO, "write error.\n", 13);
            close(file_fd);
            return 1;
        }
        if (write_all(file_fd, "\n", 1) == -1) {
            write_all(STDERR_FILENO, "write error.\n", 13);
            close(file_fd);
            return 1;
        }
    }

    break;
}

for (ssize_t i = 0; i < bytes; i++) {
    char symbol = buffer[i];

    if (symbol == '-') {
        if (in_number || sign_seen || after_dot) {
            invalid_line = 1;
        } else {
            sign = -1;
            sign_seen = 1;
        }
    }

    } else if (symbol == '.') {

        if (after_dot) {
            invalid_line = 1;
        } else {
            after_dot = 1;
        }
    }

    } else if (symbol >= '0' && symbol <= '9') {

        int digit = symbol - '0';

        if (!after_dot) {
            current_number = current_number * 10.0f + digit;
        } else {
            current_number += digit / fraction_divisor;

            fraction_divisor *= 10.0f;
        }

        in_number = 1;
    } else if (symbol == ' ' || symbol == '\n') {

        if ((sign_seen || after_dot) && !in_number) {
            invalid_line = 1;
        }

        if (in_number) {
            sum += current_number * sign;

```

```

        line_has_number = 1;
        sign = 1;
        current_number = 0.0f;
        in_number = 0;
        sign_seen = 0;

        after_dot = 0;
        fraction_divisor = 10.0f;
    }

    if (symbol == '\n') {

        if (invalid_line) {
            write_all(STDERR_FILENO, "Invalid input.\n", 15);
        } else if (line_has_number) {
            if (write_float(file_fd, sum) == -1) {
                write_all(STDERR_FILENO, "write error.\n", 13);
                close(file_fd);
                return 1;
            }
            if (write_all(file_fd, "\n", 1) == -1) {
                write_all(STDERR_FILENO, "write error.\n", 13);
                close(file_fd);
                return 1;
            }
        }

        sum = 0.0f;
        current_number = 0.0f;

        sign = 1;
        sign_seen = 0;
        in_number = 0;

        after_dot = 0;
        fraction_divisor = 10.0f;

        line_has_number = 0;
        invalid_line = 0;
    }

}

else {
    invalid_line = 1;
}

}

}

if (close(file_fd) == -1) {
    write_all(STDERR_FILENO, "close error.\n", 13);
    return 1;
}

return 0;
}

```

Протокол работы программы

```

Child receHello
deesutka@MacBook-Pro-deesutka first_OS % gcc parent.c -o parent
gcc child.c -o child
deesutka@MacBook-Pro-deesutka first_OS % ./parent
Child received: Hello
deesutka@MacBook-Pro-deesutka first_OS % gcc parent.c -o parent
gcc child.c -o child
deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: result.txt
Filename: result.txtRead:Hello
deesutka@MacBook-Pro-deesutka first_OS % gcc parent.c -o parent
gcc child.c -o child

./parent
Enter the filename: result.txt
Enter number: Filename: result.txt1.5 2.5 3
Read:1.5 2.5 3

deesutka@MacBook-Pro-deesutka first_OS % gcc parent.c -o parent
gcc child.c -o child

./parent
Enter the filename: result.txt
Enter number:
Filename: result.txt1.5 2.5 3
Read:1.5 2.5 3

deesutka@MacBook-Pro-deesutka first_OS % gcc parent.c -o parent
gcc child.c -o child

./parent
Enter the filename: resu
Enter number:
Filename: resu^C
deesutka@MacBook-Pro-deesutka first_OS %
* Журнал восстановлен

• deesutka@MacBook-Pro-deesutka first_OS % gcc -std=c11 -Wall -Wextra -Wpedantic parent.c -o parent
gcc -std=c11 -Wall -Wextra -Wpedantic child.c -o child
• deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: result.txt
Enter number: (Ctrl+D)
1 2 3
• deesutka@MacBook-Pro-deesutka first_OS % cat result.txt
6.000000
• deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: float.txt
Enter number: (Ctrl+D)
1.5 2.5 3
10 -4.5 2
0.1 0.2 0.3 0.4
• deesutka@MacBook-Pro-deesutka first_OS % cat float.txt
7.000000
7.500000
1.000000
• deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: negative.txt
-1 -2 -3
5 -10 2
0
10 -10Enter number: (Ctrl+D)
^C
• deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: negative.txt
Enter number: (Ctrl+D)
-1 -2 -3
5 -10 2
0
10 -10
• deesutka@MacBook-Pro-deesutka first_OS % cat negative
cat: negative: No such file or directory
• deesutka@MacBook-Pro-deesutka first_OS % cat negative.txt
-6.000000
-3.000000
0.000000
0.000000
• deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: spaces.txt
Enter number: (Ctrl+D)
1 2 3
10 20 30
• deesutka@MacBook-Pro-deesutka first_OS % cat spaces.txt
6.000000
60.000000

```

Терминал

```

deesutka@MacBook-Pro-deesutka first_OS % cat spaces.txt
60.000000
● deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: bad1.txt
Enter number: (Ctrl+D)
1.2.3 5
Invalid input.
● deesutka@MacBook-Pro-deesutka first_OS % cat bad1.txt
● deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: bad2.txt
Enter number: (Ctrl+D)
--5 10
Invalid input.
● deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: bad3.txt
Enter number: (Ctrl+D)
12-3
Invalid input.
● deesutka@MacBook-Pro-deesutka first_OS % cat bad3.txt
● deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: bad4.txt
Enter number: (Ctrl+D)
-
Invalid input.
● deesutka@MacBook-Pro-deesutka first_OS % cat bad4.txt
● deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: bad5.txt
Enter number: (Ctrl+D)
.
Invalid input.
● deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: bad6.txt
Enter number: (Ctrl+D)
1.5 abc 3
Invalid input.
^C
● deesutka@MacBook-Pro-deesutka first_OS % cat bad6.txt
● deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: mixed.txt
Enter number: (Ctrl+D)
1.2.3
Invalid input.
1 2 3
● deesutka@MacBook-Pro-deesutka first_OS % cat mixed.txt
6.000000
● deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: eof.txt
Enter number: (Ctrl+D)
10 20 5
● deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: eof.txt
Enter number: (Ctrl+D)
10 20 5
● deesutka@MacBook-Pro-deesutka first_OS % cat eof.txt
35.000000
● deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: empty.txt
Enter number: (Ctrl+D)
1 2 3

4 5 6

7
● deesutka@MacBook-Pro-deesutka first_OS % cat empty.txt
cat: empty.txt: No such file or directory
● deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: empty.txt
Enter number: (Ctrl+D)
1 2 3

4 5 6

7
● deesutka@MacBook-Pro-deesutka first_OS % cat empty.txt
6.000000
15.000000
7.000000
● deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: zero.txt
Enter number: (Ctrl+D)
0

```

```

deesutka@MacBook-Pro-deesutka first_OS % cat parent.txt
cat: parent.txt: No such file or directory
deesutka@MacBook-Pro-deesutka first_OS % cat zero.txt
0.000000
0.000000
0.000000
deesutka@MacBook-Pro-deesutka first_OS % mv child child_backup
deesutka@MacBook-Pro-deesutka first_OS % ./parent
Enter the filename: zz.txt
Enter number: (Ctrl+D)
execve error.

deesutka@MacBook-Pro-deesutka first_OS % mv child_backup child
deesutka@MacBook-Pro-deesutka first_OS % gcc -std=c11 -Wall -Wextra -Wpedantic parent.c -o parent
deesutka@MacBook-Pro-deesutka first_OS % gcc -std=c11 -Wall -Wextra -Wpedantic parent.c -o parent
mv child_backup child
mv: child_backup: No such file or directory
deesutka@MacBook-Pro-deesutka first_OS % mv child child_backup
./parent
Enter the filename: zz.txt
Enter number: (Ctrl+D)
execve error.
Child finished with error.
deesutka@MacBook-Pro-deesutka first_OS % mv child_backup child
deesutka@MacBook-Pro-deesutka first_OS % 

```

Вывод

В ходе лабораторной работы я научился создавать дочерние процессы с помощью `fork()` и запускать в них отдельные программы с использованием `execve()`. Также я освоил передачу данных между процессами через неименованные каналы `pipe` и перенаправление стандартного ввода с помощью `dup2()`. В процессе работы я научился использовать системные вызовы `read()`, `write()`, `open()`, `close()` и `waitpid()`, а также обрабатывать возникающие при их выполнении ошибки. Основные трудности были связаны с корректной передачей данных через канал, обработкой чисел типа `float` и правильным завершением взаимодействующих процессов.